

实验6：多进程机制与系统调用

概述

项目	说明
实验编号	Lab 6 of 8
框架目录	whu-oslab-skeleton/
本实验涉及文件	user/usys.S kernel/trap/trap.c kernel/syscall/syscall.c kernel/syscall/sysproc.c kernel/proc/proc.c kernel/include/proc.h
验收标准（阶段一）	单进程用户程序通过系统调用成功打印信息并退出，屏幕出现由用户态发起的输出
验收标准（阶段二）	多进程调度正常运行， sys_fork / sys_wait / sys_exit 测试通过，屏幕出现父子进程的并发输出

框架文件速查

本实验任务	对应文件	TODO 标注
任务1：用户态桩代码	user/usys.S	TOD0 [Lab6-任务1]
任务2：陷阱分发	kernel/trap/trap.c	TOD0 [Lab6-任务2]
任务3：系统调用分发器	kernel/syscall/syscall.c	TOD0 [Lab6-任务3-步骤1,2]
任务4：参数提取辅助函数 (新增函数)	kernel/syscall/syscall.c	TOD0 [Lab6-任务4]
任务5：基础系统调用实现	kernel/syscall/sysproc.c	TOD0 [Lab6-任务5]
任务6：多进程生命周期	kernel/proc/proc.c + kernel/include/proc.h	TOD0 [Lab6-任务6]
任务7：多进程系统调用扩展	kernel/syscall/sysproc.c	TOD0 [Lab6-任务7]

第一节：从裸机调用到系统调用——跨越用户与内核的边界

在 Lab5 中，你已经建立了第一个用户态进程 `proczero`，它执行了两次 `ecall` 指令，但内核只是打印了“捕获到系统调用”就直接返回——内核还不知道用户到底在请求什么。

本实验的目标，就是把这套“接线”真正接通。

回顾一下此前的状态：用户程序发出 `ecall` → 内核的 `usertrap()` 捕获到 `scause == 8` → 你打印一行信息 → `usertrapret()` 返回。现在，我们需要在这个捕获和返回之间，建立完整的系统调用分发和执行机制。

整个实验分为两个阶段：

- **阶段一（任务1–5）**：在单进程环境下，建立完整的系统调用框架，实现 `sys_getpid`、`sys_write`、`sys_exit` 等功能较小的调用。
- **阶段二（任务6–7）**：实现完整的进程生命周期管理（fork/wait/exit 的内核逻辑），并将其包装成系统调用，使多进程真正运转起来。

第二节：理论基础——系统调用的完整生命周期

2.1 一次 `write(fd, buf, len)` 背后发生了什么？

让我们追踪用户程序调用 `write()` 时，系统内部的完整流程。

请在阅读以下内容时，同步阅读 xv6 源码中的 `user/usys.pl`、`kernel/syscall.c`、`kernel/sysproc.c` ——找到每个步骤在哪个文件的哪一行体现。

```
1 ① 用户程序
2     write(fd, buf, len)
3     ↓
4 ② 用户态桩代码 (usys.S)
5     li a7, SYS_write    // 将系统调用号存入a7
6     ecall               // 触发陷阱，硬件接管
7     ↓
8 ③ RISC-V 硬件（自动执行，不需要软件干预）
9     sepc ← 当前 PC (ecall 指令地址)
10    scause ← 8 (U-Mode ecall 的编号)
11    PC ← stvec (Lab5 设置的陷阱入口)
12    ↓
13 ④ 陷阱汇编入口 (kernelvec.S / usertrap 入口)
14    保存全部用户寄存器到 trapframe
15    ↓
16 ⑤ usertrap() (kernel/trap/trap.c)
17    识别 scause == 8
18    p->trapframe->epc += 4    // 跳过 ecall，避免死循环
19    intr_on()                 // 系统调用期间允许被打断
20    syscall()                 // 交给分发器
21    ↓
22 ⑥ syscall() (kernel/syscall/syscall.c)
23    num = p->trapframe->a7    // 读取系统调用号
24    syscalls[num]()           // 查函数指针表，调用
```

```

25         ↓
26 ⑦ sys_write() (kernel/syscall/sysproc.c)
27     argint(0, &fd)           // 提取参数: fd
28     argaddr(1, &buf)         // 提取参数: buf 用户虚拟地址
29     argint(2, &len)          // 提取参数: len
30     // ... 执行实际写操作 ...
31     return n;                // 返回值
32         ↓
33 ⑧ 返回值传递与恢复用户态
34     p->trapframe->a0 = return_value
35     usertrapret() → sret → U-Mode
36         ↓
37 ⑨ 用户程序
38     拿到 a0 作为 write() 的返回值

```

读 xv6 源码时的关键问题：

1.

xv6 的 `usys.pl` 是做什么用的？它生成的汇编与我们用宏手写的有什么等价关系？

2.

`syscall.c` 里的函数指针数组 `syscalls[]` 是如何索引的？它的大小上限是什么决定的？

3.

为什么 `usertrap()` 要在调用 `syscall()` 之前执行 `intr_on()`？`syscall()` 内部能再次调用 `intr_on()` 吗？

2.2 参数传递约定：如何安全地从用户态取出参数？

这是系统调用中最容易出错的地方，也是许多 bug 的根源。必须深刻理解。

为什么不能直接用用户传来的指针？

用户程序和内核使用不同的页表（即使当前简化版仍使用内核恒等映射，恶意的用户程序也可能传入内核地址区间内的指针）。如果内核直接解引用用户传来的地址，轻则产生未定义行为，重则造成内核安全漏洞。

正确的做法是：通过页表翻译将用户虚拟地址转为物理地址，然后安全拷贝到内核缓冲区：

```

1  用户虚拟地址 (src_va)
2      ↓ walkaddr(pagetable, va0) → 物理地址 pa0
3      ↓ memmove(kernel_buf, pa0 + offset, n) ← 安全拷贝
4  内核缓冲区

```

参数提取辅助函数族（你需要自行实现）：

xv6 设计了一套清晰的辅助函数层级，其调用关系如下：

```

1  argraw(n)          ← 最底层：直接从 trapframe->a0~a5 读取第 n 个原始值
2  └─ argint(n, *ip)  ← 取整数，调用 argraw

```

```

3 |   └─ argaddr(n, *ap)    ← 取地址 (uint64) , 调用 argraw
4 |   └─ argstr(n, buf, max) ← 取字符串, 需要 copyin 安全拷贝

```

打开 xv6 源码中的 `kernel/syscall.c`，重点阅读 `argraw`、`argint`、`argaddr`、`argstr` 这四个函数，理解每一层的实现思路，然后在本框架中参照其设计自行实现。不要照抄代码——理解它在做什么，然后用你自己的方式写出来。

关键思考（动手前先回答）：

1.

为什么 `argstr` 比 `argint` 复杂得多？字符串和整数在内存表示上的本质区别是什么？

2.

`argstr` 里的最大长度参数 `max` 是为了保护谁？如果没有这个参数限制，会发生什么？

3.

`copyin` 函数需要哪些参数才能完成安全拷贝？页表的角色是什么？（提示：阅读 `walkaddr` 的实现）

2.3 进程生命周期：fork/exit/wait 三件套

在单进程系统中，系统调用只是内核的“文秘服务”。真正让操作系统活起来的是多进程，而多进程的核心是三个资源管理操作：

系统调用	内核职责	返回语义
<code>fork()</code>	复制当前进程，创建子进程	父进程得到子进程 pid；子进程得到 0
<code>exit(n)</code>	终止当前进程，释放资源，通知父进程	不返回
<code>wait(status)</code>	等待某个子进程退出，回收其残余资源	返回退出的子进程 pid

这三个调用必须配套工作，否则会导致：

- 没有 `wait`：子进程退出后变成**僵尸进程**（资源无法回收，进程表泄露）
- 没有 `exit`：进程永远运行，调度器无法释放 CPU
- 没有 `fork`：整个系统只有一个进程（单进程模式）

状态机转换图（必须理解后再动手）：

```

1 | 分配 PCB
2 |   ↓ allocproc()、userinit()
3 | TASK_READY
4 |   ↓ scheduler() 选中
5 | TASK_RUNNING
6 |   ↓ 时钟中断 → yield()

```

```

7  TASK_READY          ← 放回就绪队列，等待下次调度
8
9  TASK_RUNNING
10  ↓  exit()
11  TASK_ZOMBIE         ← 等待父进程 wait() 来回收
12  ↓ 父进程 wait()
13  (PCB 被释放)

```

读 xv6 源码时的关键问题：

1.

xv6 的 `exit()` 函数在进程退出时一定要做哪几件事？为什么不能直接 `kfree(PCB)`？

2.

`wait()` 如果找不到子进程（或子进程还没 `exit`），它是如何“暂时搁置”自己的？（提示：查看 `sleep()` 调用）

3.

`fork()` 复制的是物理内存的哪些部分？`trapframe` 需要复制吗？复制 `trapframe` 的目的是什么？

第三节：框架结构概览与准备工作

3.1 文件结构

在开始之前，了解本实验涉及的文件结构：

```

1  whu-oslab-skeleton/
2  └─ user/
3     └─ usys.S          ← 任务1：用户态桩代码（ecall 跳板）
4  └─ kernel/
5     └─ include/
6         └─ proc.h      ← 【必读】进程数据结构定义——本实验需扩展！
7         └─ defs.h      ← 【必读】所有内核函数声明
8     └─ trap/
9         └─ trap.c      ← 任务2：usertrap() 中识别并分发 ecall
10    └─ syscall/
11        └─ syscall.c    ← 任务3,4：syscall() 分发器 + 参数辅助函数
12        └─ sysproc.c   ← 任务5,7：sys_xxx 具体实现
13    └─ proc/
14        └─ proc.c       ← 任务6：fork/exit/wait 内核逻辑

```

3.2 关键准备：检查并扩展 `struct proc`

在开始任何任务之前，请先阅读 `kernel/include/proc.h` 中的 `struct proc` 定义。你会发现，当前框架中的 `struct proc` 是为 Lab5（单进程调度）设计的，缺少实现多进程生命周期所需的若干字段。

请思考并回答：`fork/exit/wait` 机制需要哪些额外信息？

需要的语义	需要什么字段？	字段类型建议
子进程退出时，父进程能找到它	父进程指针	<code>struct proc *parent</code>
父进程 <code>wait()</code> 能读取子进程退出码	退出状态	<code>int xstate</code>
<code>wait()</code> 阻塞时能被 <code>exit()</code> 准确唤醒	睡眠通道	<code>void *chan</code>
<code>exit()</code> 能强制终止其他失控进程	被杀标志	<code>int killed</code>

你需要在 `proc.h` 的 `struct proc` 中自行添加这些字段，然后在 `proc.c` 中使用它们。这是本实验数据结构设计的第一步。

设计问题：`wait_lock`（保护 `wait/exit` 竞争）应该是 `struct proc` 内部的字段，还是一个全局变量？两种方案各有什么权衡？（参考 xv6 的设计选择）

3.3 系统调用号的管理

注意到 `user/usys.S` 和 `kernel/syscall/syscall.c` 中各自用 `#define` 重复定义了一套系统调用号常量。这是一个潜在的维护隐患——如果两处不一致，系统调用会静默失败。

思考：如何重构才能让系统调用号只定义一次？（提示：考虑新建 `kernel/include/syscall_nr.h`，然后让两处都 `#include` 它）。本实验不强制要求，但这是一个很好的工程实践。

第四节：阶段一任务实施——系统调用框架

任务1 / `user/usys.S` — 用户态桩代码

理解目标：用户程序调用的 `write()` 最终是一个普通 C 函数。这个函数是从哪里来的？它怎么变成了一个 `ecall` 指令？

答案就在 `usys.S`。这里定义的每个“系统调用桩”都极其简单：把系统调用号装入 `a7`，然后 `ecall`。用户库（`lib/`）将这些汇编符号导出为 C 函数名，C 程序就能直接调用了。

目标文件：`user/usys.S`

找到 `TODO [Lab6-任务1]`。框架已提供了 `getpid` 和 `exit` 的完整实现作为示例，也定义了 `SYSCALL` 宏（如果框架使用宏的话）。

你需要：

1. 理解 `getpid` 桩的展开逻辑：`.global`、`li a7`、`ecall`、`ret` 各自的作用。
2. 参照 `getpid`，为 `write` 补全其汇编桩（一个 `TODO` 已标注）。
3. 一并补全后续阶段需要的 `fork`、`wait` 桩。

思考：

-

函数参数（如 `write` 的 `fd`、`buf`、`len`）由谁、在何时、放到哪些寄存器？是你在汇编桩里手动放的，还是 C 调用约定自动安排的？

-

`ecall` 执行后，返回值最终从哪个寄存器传回给调用者？整个传递链条经过哪几步？

验证方法：编译后用 `riscv64-unknown-elf-objdump -d kernel.elf | grep -A5 "getpid"` 确认这些函数出现在最终镜像中。

任务2 / `kernel/trap/trap.c` — 识别并处理 `ecall` 陷阱

理解目标：`usertrap()` 是 Lab5 中你实现的用户态陷阱处理函数。目前它能识别 `scause == 8` 并打印了一行信息，但没有真正“接单”——它既没有移动 PC，也没有调用分发器。现在需要补全这个分支。

目标文件：`kernel/trap/trap.c`，函数 `usertrap()`

找到 `TODO [Lab6-任务2]`。仔细阅读其上下文：框架中 `intr_on()` 和 `syscall()` 的调用已经存在，而 `TODO` 标注的是那条缺失的、使返回后不会无限重复执行 `ecall` 的关键操作。

你需要在调用 `syscall()` 之前，完成一个关键步骤。思考：

- 如果 `sret` 后 PC 仍然指向 `ecall` 指令地址，会发生什么？
- 通过哪个字段可以修改“返回用户态后从哪里继续执行”？

深入思考：

-

步骤中的 `intr_on()` 如果省略会怎样？在何种内核场景下省略它反而更安全？

-

时钟中断也可能进入 `usertrap()`（因为 `mideleg` 委托），此时 `scause != 8`，你的代码如何区分并分别处理？

任务3 / `kernel/syscall/syscall.c` — 系统调用分发器

理解目标：`syscall()` 是“总机接线员”——它从 `trapframe->a7` 读取调用号，在函数指针表中查找对应的内核函数，调用它，然后把返回值写回 `trapframe->a0`。

目标文件: `kernel/syscall/syscall.c`

步骤1 (`TOD0 [Lab6-任务3-步骤1]`) : 构建系统调用注册表。

当前 `syscalls[]` 数组是空的 (所有条目为 `NULL`)。你需要:

1. 在文件顶部用 `extern` 声明来自 `sysproc.c` 的各个 `sys_xxx` 函数。
2. 在 `syscalls[]` 数组中, 用指定初始化 (designated initializer) 语法填入各函数指针。

阅读 xv6 的 `kernel/syscall.c` 中 `syscalls[]` 的定义方式。注意 xv6 使用的语法风格, 理解为什么使用 `[SYS_xxx] = sys_xxx` 这种写法。

思考: 使用 `[SYS_xxx] = ...` 这种指定初始化写法, 和按顺序顺次填写相比, 有什么优势? 如果系统调用号之间有空洞 (比如 1、2、3 之后跳到 11), 两种写法会有什么不同结果?

步骤2 (`TOD0 [Lab6-任务3-步骤2]`) : 实现 `syscall()` 函数体。

目前 `syscall()` 函数体为空。你需要:

1. 从当前进程的 `trapframe` 中读取系统调用号 (思考: 存在哪个寄存器字段?)。
2. 判断该调用号是否合法 (越界检查、空指针检查)。
3. 合法则调用对应函数, 将返回值写回 `trapframe->a0`; 非法则打印错误消息。

参照 xv6 的 `syscall()` 函数理解完整逻辑后, 在你的框架中自行设计并实现, 不要照搬代码。

任务4 / `kernel/syscall/syscall.c` — 参数提取辅助函数

理解目标: 系统调用的参数按照 RISC-V 调用约定存放在 `a0 - a5` (也就是 `trapframe` 的对应字段)。你需要实现一套安全提取它们的辅助函数, 供 `sysproc.c` 中的 `sys_xxx` 函数使用。

目标文件: `kernel/syscall/syscall.c` (在文件末尾新增函数, 并在 `defs.h` 中添加声明)

找到 `TOD0 [Lab6-任务4]` (如果框架未标注, 在 `syscall.c` 末尾自行添加)。

你需要实现以下四个函数, 参数层级如第 2.2 节所述:

- `argraw(n)` — 从 `trapframe` 读取第 `n` 个参数的原始 `uint64` 值
- `argint(n, *ip)` — 读取整数参数 (调用 `argraw`)
- `argaddr(n, *ap)` — 读取地址参数 (`uint64`) (调用 `argraw`)
- `argstr(n, buf, max)` — 读取字符串参数 (需要安全拷贝!)

实现方法: 先在 xv6 的 `kernel/syscall.c` 中找到这四个函数, 逐行阅读理解其实现逻辑, 然后关闭 xv6 源码, 合上书本, 自己写出等价实现。

特别注意 `argstr` 需要调用 `copyin` 或 `copyinstr` ——这两个函数在本框架的 `vm.c` 中是否已经实现? 如果没有, 你需要先参照 xv6 的 `vm.c` 在框架中实现它们, 或者采用简化方案 (例如, 在当前简化的恒等映射地址空间下, 用户虚拟地址等于物理地址, 可以临时直接拷贝, 但需在报告中说明这种简化的安全限制)。

关键思考:

- 如果用户传了一个长度为 100000 字节的字符串, 但你的内核 `buf` 只有 256 字节, `argstr` 是否会导致内核缓冲区溢出? `max` 参数如何防止这类攻击?

当字符串跨越两个物理页边界时（如字符串从 `0xffff` 开始），`copyin` 如何处理？（提示：看 xv6 `copyin` 循环的边界条件）

任务5 / `kernel/syscall/sysproc.c` — 基础系统调用实现

有了参数提取工具，现在可以实现真正有功能的系统调用了。

⚠ 注意框架中的描述错误：框架 `sysproc.c` 中 `sys_write` 的 TODO 注释里写的是 `sys_exit` 的实现步骤，这是一个复制粘贴错误。请以下面的手册描述为准，忽略 TODO 注释中关于“打印退出信息”和“设置 TASK_ZOMBIE”的步骤——那是 `sys_exit` 的职责，不是 `sys_write`。

📁 目标文件： `kernel/syscall/sysproc.c`

找到 `TODO [Lab6-任务5]`，实现以下三个系统调用：

`sys_getpid` — 获取当前进程号

最简单的系统调用，不需要任何参数。

思考：你如何在内核中获得“当前正在运行的进程”的指针？（提示：已有函数可用）然后从该指针中取出什么字段即可？

- 验收测试：用户程序调用 `getpid()` 应返回 1（第一个进程的 pid）。

`sys_write` — 向内核请求输出

简化版实现：从参数中提取字符串并调用内核输出函数。

你需要完成的工作：

1. 用任务4实现的参数函数，从 `trapframe` 中安全地提取字符串到内核缓冲区
2. 将提取到的字符串通过 `printf` 或逐字符 `uart_putc` 输出到串口

设计问题：

- `fd`（第0个参数）在本简化实现中如何处理？未来 Lab7 实现文件系统后，这里应该做什么？
- 内核缓冲区应分配多大比较合适？如何防止用户传入超长字符串导致栈溢出？

- 验收测试：用户程序调用 `write(1, "Hello from user!\n", 17)` 后，屏幕出现该字符串。

`sys_exit` — 终止进程

阅读 xv6 的 `sys_exit` 实现（非常简短），理解其参数提取方式和它调用的内核函数。

注意：此阶段 `exit()` 内核函数还未完整实现（任务6），你可以先用一个简化版本（直接让进程变成 ZOMBIE 状态并切回调度器），在任务6完成后再替换为完整实现。

阶段一验收：修改 `user/proczero.c` 或 `user/initcode.S`，让用户程序调用 `getpid()` 和 `write()` 打印信息后调用 `exit(0)`。运行 `make run`，屏幕应出现：

```
1 Hello from user!    (← 来自用户程序的输出)
2 Process 1 exited with status 0    (← 内核的退出日志)
```

第五节：阶段二理论——进程同步的必要性

在进入 `fork/exit/wait` 的实现之前，有一个问题必须先回答：**单核系统，还需要同步吗？**
答案是：**需要**。本框架虽然设置了 `NCPU = 1`，但时钟中断可以在任意指令之间触发，这意味着内核代码随时可能被“打断”——这在功能上等价于并发，产生竞态条件一样严峻。

5.1 竞态条件从哪里来？

观察 `proc.c` 中的 `allocpid()`：

```
1 int allocpid(void) { return nextpid++; }
```

`nextpid++` 在汇编层面是三步操作：**读 → 加 → 写**。如果 `fork()` 在“读”和“写”之间被时钟中断打断，调度器切换到另一个进程也在 `fork()`，两个进程就会得到**相同的 pid**——这是严重的状态损坏。
类似的竞态危险存在于：

危险操作	竞态后果
<code>nextpid++</code>	两个进程得到相同 pid
修改 <code>p->state</code> (RUNNING→READY)	调度器可能同时把同一进程调度到两个动作
遍历 <code>proc[]</code> 查找 ZOMBIE 子进程	<code>exit()</code> 同时修改状态，遍历看到中间状态
<code>wait()</code> 检查无子进程后决定睡眠	子进程在检查后立即 <code>exit()</code> ， <code>wakeup</code> 先于 <code>sleep</code> ，导致 丢失唤醒

5.2 自旋锁 (Spinlock)

自旋锁是内核中最基础的同步原语。它的本质是：**在等锁时不睡眠，而是原地忙等（自旋）**。
为什么内核不直接用互斥锁（睡眠锁）而用自旋锁？
互斥锁在等待时会让进程睡眠，睡眠需要调用调度器，调度器本身也需要锁——形成死锁。自旋锁不睡眠，持锁时间极短，是保护内核短临界区的最佳选择。
自旋锁的实现核心：原子交换 (Test-and-Set)
普通的“读-检查-写”三步操作不是原子的，可以被中断打断。正确的自旋锁必须用 CPU 提供的原子指令一步完成“读-改-写”操作，对应 RISC-V 下的 `amoswap` 指令系列。
持有自旋锁时必须关中断！ 这是本框架面临的关键约束：

```
1 进程A 持有 proc_lock, 正在修改进程表
```

```
2   → 时钟中断触发
3   → 调度器 (也需要 proc_lock)
4   → 死锁! 调度器无法获取 proc_lock, 进程A也无法运行去释放它
```

因此 `acquire(lock)` 必须先关中断, `release(lock)` 恢复中断。

阅读 xv6 源码: 打开 xv6 的 `kernel/spinlock.c`, 逐行理解 `acquire()`、`release()`、`push_off()`、`pop_off()` 的实现。重点关注:

- 为什么 `push_off` 而不是直接 `intr_off`? (提示: 锁的嵌套与中断嵌套的关系)
- `_sync_synchronize()` 的作用是什么? 如果去掉它, 会出现什么问题?

5.3 本框架的同步策略

查看 `include/proc.h`: 当前框架的 `struct proc` 没有 `lock` 字段, 你在任务6时需要自行决定同步策略, 有两种合理的方案:

方案A (简化版, 推荐入门): 在关键操作前后直接关/开中断

在修改进程表、进程状态等临界区前后调用 `intr_off()` / `intr_on()` 保护。

优点: 实现简单, 无死锁风险。缺点: 粒度粗, 关中断期间时钟信号也被屏蔽, 系统抖动增大。适合本实验的单核环境。

方案B (完整版, 推荐有余力的同学): 为 `struct proc` 添加自旋锁字段

参照 xv6 的 `spinlock.c`, 将自旋锁引入框架, 在 `fork()/exit()/wait()` 中按照 xv6 的模式使用锁保护临界区。

思考 (动手前回答):

1.

方案A中, `fork()` 分配子进程 PCB 后立刻将其加入就绪队列, 但此刻复制用户页尚未完成——时钟中断来了会怎样? 方案A能防止这种情况吗? 你打算如何解决?

2.

方案B中, `wait()` 持有父进程锁等待时, 子进程在 `exit()` 中也想获取自己的锁——如何设计锁的获取顺序, 避免死锁? (参考 xv6 关于锁顺序的注释)

5.4 sleep/wakeup 与“丢失唤醒”问题

`wait()` 需要在没有子进程退出时睡眠等待, 等子进程 `exit()` 时再被唤醒。这套 `sleep/wakeup` 机制听起来简单, 但有一个著名的陷阱:

场景:

```
1 父进程 wait():
2    1. 遍历进程表, 没有 ZOMBIE 子进程
3    2. 决定睡眠...
4        ← 时钟中断! 子进程抢到 CPU 并执行 exit()
5        ← 子进程调用 wakeup(parent) ← wakeup 比 sleep 先执行!
6    3. 父进程继续: sleep(parent)    ← 父进程永远睡眠!
```

这叫**丢失唤醒 (Lost Wakeup) **问题。xv6 的解决方案是: 把“检查条件”和“进入睡眠”放在同一把锁的保护下, 使检查与睡眠之间不存在可被打断的窗口。

阅读 xv6 的 `sleep()` 和 `wakeup()` 函数, 理解:

- `sleep(chan, lock)` 需要两个参数，为什么要传入一把锁？这把锁在 `sleep` 内部如何被处理？
- 被 `wakeup` 唤醒后，`sleep` 返回时是否仍持有该锁？

注意：本框架 `defs.h` 已声明了 `sleep(void *chan)` 和 `wakeup(void *chan)`，但尚未实现。你需要在 `proc.c` 中实现它们。`sleep` 的签名只有一个参数 `chan`，如果你选择方案B（带锁），可以扩展其签名以支持锁参数（需同步修改 `defs.h`）；如果选择方案A，可在 `sleep` 内部配合 `intr_off` 实现。

第六节：阶段二理论——多进程的资源管理

完成阶段一且理解同步机制后，现在进入 `fork/exit/wait` 的实现原理。

本节介绍内核实现原理。请在理解后再动手。

6.1 `fork()` — 进程的“分裂复制”

`fork()` 是操作系统中最精妙的设计之一：调用一次，返回两次。

它在内核中需要完成以下工作（具体步骤请参照 xv6 的 `fork()` 函数理解，然后自行设计实现）：

- 1 `fork()` 内核实现的核心逻辑（以流程描述，非代码）：
- 2
- 3 1. 分配新 PCB：找到空槽位，分配 pid，分配 trapframe
- 4 2. 复制用户地址空间：为子进程分配物理页，逐页拷贝父进程内容
- 5 3. 复制 trapframe：使子进程从 `sret` 返回时处于与父进程相同的位置
- 6 4. 让子进程“返回 0”：在子进程的 trapframe 中修改某寄存器的值
- 7 5. 建立父子关系：记录 parent 指针
- 8 6. 让子进程进入就绪队列：设置状态为 `TASK_READY`

关键问题：

- 父子进程的 trapframe 完全相同（包括 `epc`），它们会从同一个地址开始执行。但父进程返回 pid，子进程返回 0——这个差异是由谁、在何时、通过修改什么设置的？
- 复制用户地址空间时，如果父进程有多个页面（代码页 + 堆 + 栈），每个页面都需要复制吗？如何找到所有需要拷贝的页？

6.2 `exit()` — 进程的“优雅离开”

进程不能粗暴地自我销毁，因为它的资源不能在自己运行时释放（你不能在自己脚下挖洞）。

`exit()` 的职责（参照 xv6 的 `exit()` 函数，理解每一步的必要性）：

- 1 `exit(n)` 内核实现的要点：
- 2

- 3 1. 保存退出状态 (n) 到 PCB 的 xstate 字段
- 4 2. 处理孤儿子进程：遍历进程表，找到自己的子进程
- 5 → 将它们的 parent 移交给 init 进程（处理孤儿的“孤儿院”机制）
- 6 3. 唤醒等待中的父进程
- 7 4. 将自己状态变为 TASK_ZOMBIE（不是释放！）
- 8 5. 放弃 CPU，切回调度器（永不返回）

为什么是 ZOMBIE 而不是直接释放 PCB？

进程退出时，它的内核栈仍然是活跃的——`exit()` 本身就在它的内核栈上运行。如果此时释放内核栈，函数就会在已释放的内存上继续执行——未定义行为立即发生。ZOMBIE 状态让进程的资源留存，由父进程在 `wait()` 中代为最终清理。

6.3 `wait()` — 父进程的“资源回收员”

- ```
1 wait(&status) 内核实现的要点：
2
3 1. 扫描进程表，查找 parent == myproc() 的子进程
4 2. 情况A：找到了 ZOMBIE 子进程
5 → 安全地将退出状态写回用户传来的 &status 地址（安全拷贝！）
6 → 释放子进程的所有资源（页表、内核栈、PCB）
7 → 返回子进程 pid
8
9 3. 情况B：有子进程但都未退出
10 → 睡眠等待，让出 CPU
11 → 被 exit() 唤醒后，重新回到步骤1继续扫描
12
13 4. 情况C：没有任何子进程
14 → 返回 -1
```

### 关键问题：

- 

`wait()` 中将退出状态写回用户地址 `&status` 时，为什么不能直接 `*status = child->xstate`？应该用什么函数来安全写入？

- 

情况B中，睡眠前遍历了一遍进程表没有发现 ZOMBIE，但如何保证“扫描结束”和“进入睡眠”之间，子进程没有偷偷 `exit` 并 `wakeup`？（这就是 5.4 节“丢失唤醒”问题，你的 `wait` 实现必须解决它）

---

## 第七节：阶段二任务实施——多进程生命周期

### 任务6 / `kernel/proc/proc.c` — 实现 `fork/exit/wait` 内核逻辑

 **目标文件：** `kernel/proc/proc.c`（以及 `kernel/include/proc.h` 扩展）

找到 `TODO [Lab6-任务6]`（框架中该文件已有 Lab5 的内容，你需要在末尾新增函数）。

**在开始写代码之前，务必完成以下准备工作：**

1. 确认你已在 `proc.h` 的 `struct proc` 中添加了 `parent`、`xstate`、`chan`、`killed` 字段（第 3.2 节）。
2. 阅读框架中已实现的 `allocproc()`、`scheduler()`、`yield()`，理解进程表的访问模式，确定你的同步策略（方案A或方案B）。
3. 实现 `sleep(chan)` 和 `wakeup(chan)` 函数（`defs.h` 已声明）。阅读 xv6 的同名函数，理解：
  - `sleep` 如何原子地“释放等待锁 + 进入睡眠状态”？
  - `wakeup` 如何遍历进程表找到所有睡在同一 `chan` 上的进程并唤醒它们？
4. 阅读 xv6 的 `fork()`、`exit()`、`wait()` 函数，对比框架与 xv6 的 `struct proc` 字段差异，确认需要哪些适配。

**关于用户内存复制（fork 中）：**

`fork()` 中最复杂的步骤是复制用户地址空间。参照 xv6 的 `uvmcopy()` 实现，或根据框架的简化映射模型（恒等映射）设计一个合适的拷贝方案。

**设计思考：**

- 如何判断父进程的用户地址空间共有多少页？`p->sz` 字段的语义是什么？
- 复制页时，子进程的物理页是新分配的，还是与父进程共享物理页？两者会有什么安全性和正确性差异？

**关键注意事项：**

- `exit()` 中将孤儿子进程移交时，需要找到 `pid=1` 的 `proczero` 进程
- 访问他人的 PCB 时，需要按照你选择的同步方案保护临界区
- `wait()` 的睡眠必须与“检查是否有 ZOMBIE 子进程”在同一保护区间内，防止丢失唤醒

## 任务7 / `kernel/syscall/sysproc.c` — 多进程系统调用包装

将任务6实现的内核函数包装成系统调用。

 **目标文件：** `kernel/syscall/sysproc.c`

找到 `TODO [Lab6-任务7]`，实现以下两个系统调用：

`sys_fork` — `fork` 的系统调用封装

参考 xv6 的 `sys_fork` 实现——这是最简短的系统调用之一，思考：它需要提取什么参数吗？为什么或为什么不？

`sys_wait` — `wait` 的系统调用封装

参考 xv6 的 `sys_wait` 实现，注意：用户传来的 `status` 是一个用户空间地址，应该如何提取？直接传给内层的 `wait()` 函数，还是需要先做什么处理？

同时，更新任务3中的 `syscalls[]` 注册表，将 `sys_fork` 和 `sys_wait` 加入其中，确保系统调用号与 `usys.S` 中的定义一致。

## 第八节：验收测试设计

修改用户态程序（`user/proczero.c`）加入以下测试：

基础验收（阶段一+二必须通过）：

```
1 // 1. 打印进程号
2 int pid = getpid(); // sys_getpid
3 // 自行编写打印 pid 的代码
4
5 // 2. fork 测试
6 int child = fork(); // sys_fork
7 if(child == 0) {
8 // 子进程分支（自行设计输出内容）
9 exit(1); // sys_exit
10 } else {
11 // 父进程分支（自行设计等待和输出逻辑）
12 int status;
13 int cpid = wait(&status); // sys_wait
14 // 打印回收结果
15 }
16
17 // 3. 正常退出
18 exit(0);
```

预期输出（参考，允许顺序不完全相同）：

```
1 My pid: 1
2 Child: hello
3 Parent: child 2 exited with 1
4 Process 1 exited with status 0
```

进阶测试（可选，加深理解）：

将 `fork()` 在循环中调用多次，让父进程等待所有子进程回收。观察输出顺序，思考为什么子进程的执行和退出顺序是不确定的。

---

## 第九节：调试与故障排除

| 症状                                                             | 优先排查                                                                                                          |
|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>sys_getpid</code><br>返回值异常（如0或负数）                        | 检查 <code>syscall.c</code> 中的 <code>p-&gt;trapframe-&gt;a0</code> 赋值是否正确；检查 <code>syscalls[]</code> 注册表是否有对应条目 |
| <code>sys_write</code><br>无输出或乱码                               | 检查 <code>argstr</code> 的安全拷贝逻辑；用户字符串是否以 <code>\0</code> 结尾；内核缓冲区是否足够大                                         |
| <code>fork</code><br>后子进程不运行                                   | 检查 <code>child-&gt;status</code> 是否设为了 <code>TASK_READY</code>                                                |
| <code>fork</code><br>后父子进程都返回0                                 | 思考：谁、在什么时候、修改了哪个字段，使父子进程得到不同的“返回值”？                                                                           |
| <code>wait</code> 永久阻塞                                         | 检查 <code>exit()</code> 是否调用了 <code>wakeup</code> ；检查睡眠和唤醒使用的 <code>chan</code> 是否一致                           |
| 系统崩溃于 <code>fork()</code><br>中的内存复制                            | 检查新页物理地址是否有效（ <code>kalloc</code> 是否成功）；<br><code>mappages</code> 的权限位是否包含 <code>PTE_U</code>                 |
| 孤儿进程无人回收                                                       | 检查 <code>exit()</code> 中是否正确将孤儿子进程移交给了<br><code>proczero</code> （pid=1 的进程）                                   |
| <code>sys_write</code><br>TODO 逻辑看起来像<br><code>sys_exit</code> | 这是框架源文件中的描述错误，以手册为准（第五节任务5有说明）                                                                                |

GDB 调试要点：

```
1 make qemu-gdb # 终端1：启动等待 GDB 连接的 QEMU
2 riscv64-unknown-elf-gdb kernel.elf # 终端2
3
4 (gdb) b syscall # 断点：系统调用分发器入口
5 (gdb) b sys_fork # 断点：fork 系统调用
6 (gdb) b exit # 断点：内核 exit 函数
7 (gdb) c
8
9 # 在 syscall() 断点处，检查调用号
10 (gdb) p p->trapframe->a7 # 应为系统调用号
11 (gdb) p p->pid # 当前进程
12
13 # 在 fork() 中，检查子进程 PCB 是否正确初始化
14 (gdb) p np->status
15 (gdb) p np->trapframe->a0 # 应为 0（子进程的"返回值"）
```

第十节：思考题（进阶验收）



1. `epc += 4` 的时机：为什么必须在进入 `syscall()` 之前（即在 `usertrap()` 中）执行 `epc += 4`，而不是在 `syscall()` 内部或各个 `sys_xxx` 函数中执行？如果放在 `sys_exit()` 里，会有什么问题？
  2. 僵尸进程的意义：`exit()` 之后进程为什么不直接销毁，而是变成 ZOMBIE 状态？如果父进程也退出了（没有调用 `wait`），谁来承接这些孤儿进程的“尸体”回收工作？
  3. `fork` 的写时复制（COW）：当前你的 `fork()` 完整拷贝了父进程的物理页。如果父进程有 100MB 数据，每次 `fork` 都需要拷贝 100MB，效率极低。Linux 如何用“写时复制”优化这个问题？从页表权限位的角度描述实现思路。
  4. 用户指针安全：如果用户程序故意在 `write()` 调用中传入内核虚拟地址（如 `0x80000000`），你的 `argstr` / `copyin` 应该如何防御这种攻击？如果没有防御，会发生什么后果？
  5. `wait` 的两层查找：POSIX 标准中 `wait()` 返回“任意一个”已退出的子进程 pid。如果父进程有多个子进程按不同顺序退出，你如何保证 `wait()` 每次都能正确找到对应的子进程并回收，而不是重复回收同一个？
  6. 系统调用号管理：`user/usys.S` 和 `kernel/syscall/syscall.c` 中分别用 `#define` 定义了一套系统调用号常量。如果维护人员只改了一处，会发生什么故障？提出并实现一个消除这种重复定义的方案。
- 

## 第十一节：实验完成后的全貌

完成本实验后，`start_main()` 的形态保持不变（Lab5 已完成）：

```
1 void start_main() {
2 kinit(); kmininit(); kminithart(); // Lab3: 内存与分页
3 trapinithart(); // Lab4: 注册陷阱向量
4 intr_on(); // Lab4: 开启 S-Mode 中断
5 procinit(); // Lab5: 初始化进程表
6 userinit(); // Lab5: 创建 proczero (含多进程测试代码)
7 scheduler(); // Lab5→Lab6: 进入调度器 (永不返回)
8 }
```

系统调用分发器现在完整运转：

```
1 用户调用 fork()
2 → usys.S: li a7, SYS_fork; ecall
3 → usertrap(): scause==8 → epc+=4 → syscall()
4 → syscall(): syscalls[SYS_fork]() → sys_fork()
5 → sys_fork(): fork() → 创建子进程
6 → trapframe->a0 = child_pid / 0
7 → usertrapret() → sret
8 用户得到返回值，父子进程从此分道扬镳
```

**Lab7 预告：**目前 `sys_write` 只能打印字符串到串口。在 Lab7（文件系统）中，你将实现真正的块设备读写、inode 管理和目录结构，届时 `sys_read`、`sys_write`、`sys_open`、`sys_close` 等调用将操纵持久化存储，操作系统从此具备了“记忆”的能力。